

Proyecto de big data e inteligencia artificial
CURSO DE ESPECIALIZACIÓN EN
INTELIGENCIA ARTIFICIAL Y BIG DATA
MODALIDAD ONLINE

FASE 1 — Documentación y Diseño Técnico

1. Título del proyecto + integrantes del equipo

Nombre del proyecto : CineMatch Sistema de recomendación películas

Integrantes y Rol :

- Badr Lahrarti – Data Engineer & Coordinador Técnico
- Alejandro González Mateos – Machine Learning Engineer
- Daniel Ruiz González – Backend & API Developer
- Francisco López Fenoll – Frontend & Visualización Developer

FASE 1 — Documentación y Diseño Técnico.....	1
1. Título del proyecto + integrantes del equipo.....	1
2. Resumen ejecutivo.....	3
3. Descripción funcional del sistema.....	3
4. Alcance definitivo (IN / OUT).....	5
5. Arquitectura del sistema (alto nivel).....	6
5.1 Diagrama de arquitectura :.....	6
5.2 Descripción de cada componente:.....	7
6. Tecnologías seleccionadas.....	8
7. Modelo de datos.....	9
Diagrama entidad-relación :.....	9
Explicación del diagrama :.....	9
Diccionario de variables :.....	10
8. Datos.....	10
8.1 Fuente o fuentes :.....	10
8.2 Licencia:.....	11
8.2 Tipo de datos :.....	11
8.3 Volumen estimado :.....	11
8.4 Frecuencia de actualización:.....	12
8.5 Calidad esperada:.....	12
8.6 Riesgos de datos (faltantes, sesgos, ruido...).....	12
9. Ética, legalidad y accesibilidad.....	14
11. Plan de trabajo.....	19
Estructura de Reuniones Semanales :.....	19
Cronograma Gantt por Semanas.....	20
12. Estrategia de desarrollo.....	22
13. Riesgos técnicos actualizados.....	23
14. Repositorio del proyecto.....	23

2. Resumen ejecutivo

CineMatch es un sistema inteligente de recomendación de películas que tiene como objetivo resolver el problema de la sobrecarga de opciones en plataformas de contenido audiovisual. Actualmente, los usuarios se enfrentan a miles de títulos disponibles, lo que genera indecisión y una experiencia poco eficiente en el descubrimiento de contenido.

El proyecto propone desarrollar un sistema basado en técnicas de filtrado colaborativo utilizando el dataset público MovieLens. A partir del historial de valoraciones de usuarios anónimos, el sistema generará recomendaciones personalizadas que permitan identificar películas relevantes para cada perfil.

El valor del proyecto radica en **aplicar técnicas reales de Inteligencia Artificial y Big Data en un entorno estructurado**, construyendo un pipeline reproducible de datos, un modelo evaluable y una arquitectura modular con backend y frontend. Esto permitirá simular un sistema real de recomendación escalable y orientado a mejorar la experiencia de usuario.

3. Descripción funcional del sistema

El sistema:

- Procesa datos históricos de ratings.
- Construye un modelo de recomendación.
- Devuelve un Top-K de películas recomendadas.
- Permite visualizar resultados en una interfaz básica.

Flujo general de uso :

1. Inicio de sesión
 2. el usuario responde a formulario de preguntas de gusto
 3. El sistema Consulta el Modelo entrenado
 4. Se generan recomendaciones personalizadas
 5. Se muestran películas sugeridas con información básica
-
1. **Inicio de sesión / selección de perfil**
El usuario accede a la aplicación y selecciona un perfil o introduce preferencias básicas.
 2. **Solicitud de recomendaciones**
El sistema recibe la información del usuario y envía la solicitud al motor de recomendación.
 3. **Consulta del modelo**
El backend consulta el modelo de recomendación previamente entrenado.
 4. **Generación de recomendaciones**
El sistema genera un conjunto de recomendaciones personalizadas (Top-K).

5. Visualización de resultados

Se muestran las películas recomendadas con información básica (título, género, puntuación).

Casos principales de usuario

- **Caso 1** - usuario existente :
 - El usuario accede a la plataforma.
 - Se identifica.
 - Usuario solicita recomendaciones.
 - El sistema genera recomendaciones basadas en sus gustos.
 - Usuario visualiza películas recomendadas con información básica.
- **Caso 2** - usuario nuevo (**cold start**) :
 - El usuario accede a la plataforma.
 - El usuario se registra en la plataforma.
 - Accede al formulario géneros de películas.
 - Usuario solicita recomendaciones.
 - El sistema genera recomendaciones basadas en sus gustos.
 - Usuario visualiza películas recomendadas con información básica.
- **Caso 3** - Consulta de recomendaciones :
 - El usuario visualiza una serie de 10 películas en carrusel una a una.
 - El usuario puede ver información básica de cada película.
 - El usuario podrá pasar a la siguiente película de forma intuitiva.
 - Una vez pasadas las 10 películas del carrusel, se le ofrece al usuario la posibilidad de que se le recomienden otras 10.

Qué puede y qué no puede hacer el sistema

Que puede :

- generar recomendaciones personalizadas
- calcular métricas de evaluación del modelo
- visualizar resultados en interfaz básica
- Simular el gusto del usuario.

Que no puede :

- integrarse con plataformas reales (netflix, prime, etc)
- Procesar datos en tiempo real
- Nuevas películas más allá de 2023 no incluidas
- retroalimentar el modelo en tiempo real.

4. Alcance definitivo (IN / OUT)

Incluye:

Funcionalidades mínimas obligatorias

- Generación de recomendaciones personalizadas de películas.
- Visualización de un listado Top-10 de películas sugeridas.
- Persistencia de datos para cada usuario en sus gustos.
- Formulario inicial de preferencias para nuevos usuarios.
- Posibilidad de cambiar gustos de usuario para recomendaciones.
- Interfaz gráfica sencilla para interactuar con el sistema.

Funcionalidades opcionales

- Interfaz gráfica compleja.
- Filtros adicionales.
- Visualización de estadísticas generales.

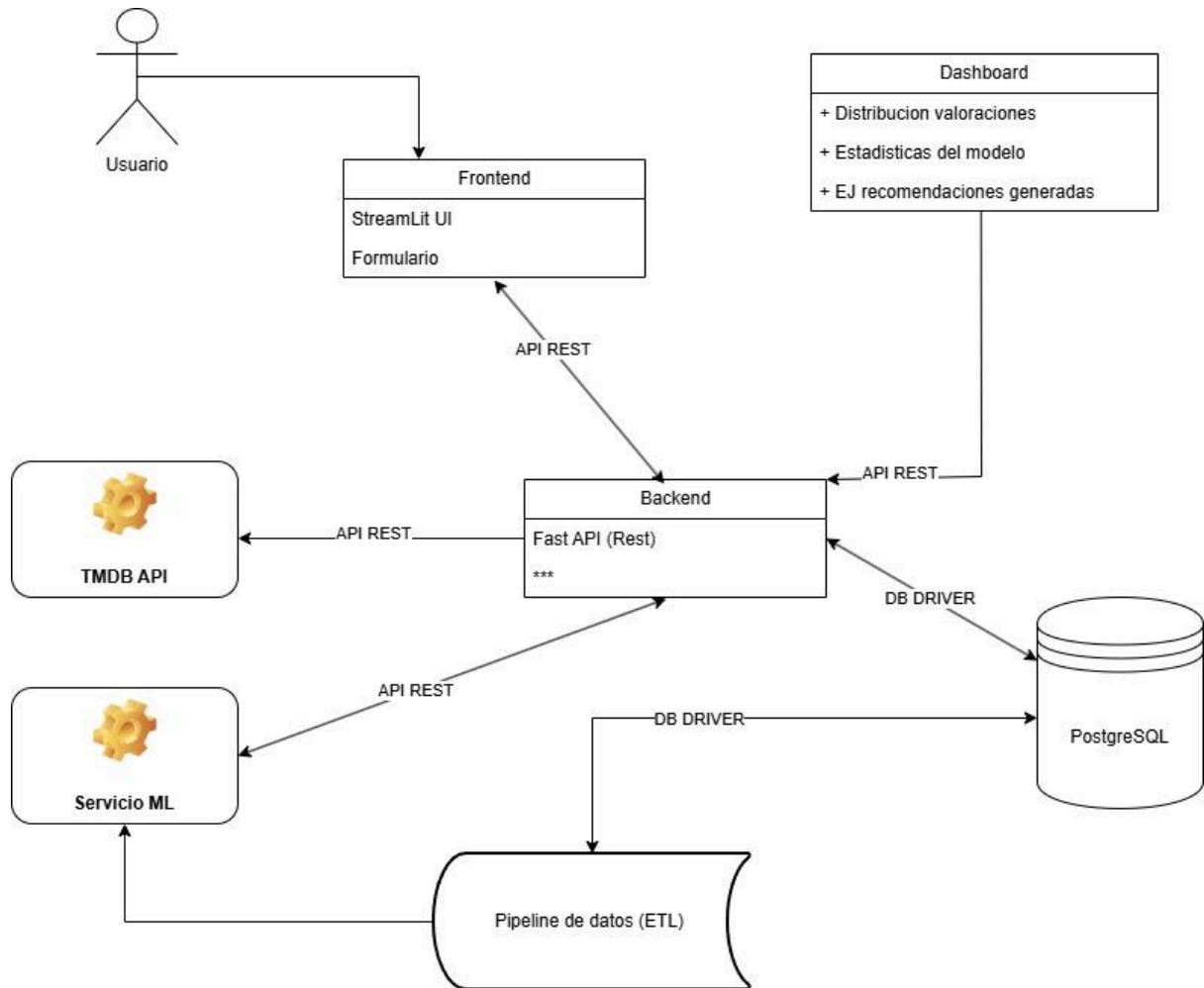
No incluye:

Elementos descartados explícitamente

- Integración con plataformas reales de streaming.
- Sistema de pagos/suscripciones.
- Gestión real de cuentas con datos personales.

5. Arquitectura del sistema (alto nivel)

5.1 Diagrama de arquitectura :



5.2 Descripción de cada componente:

Frontend

- La capa de frontend proporciona la interfaz de usuario del sistema, permitiendo al usuario interactuar con el motor de recomendación. Su función principal es facilitar el acceso a las recomendaciones personalizadas, la selección de usuario, configuración de gustos y la visualización de resultados.

Backend

- La capa backend expone una API REST mediante **FastAPI** y actúa como el núcleo del sistema, gestionando las solicitudes del FrontEnd, coordinando el acceso al modelo de Machine Learning, la base de datos y los servicios externos, y devolviendo las respuestas en formato JSON.

Base de datos

- La base de datos almacena la información necesaria para el funcionamiento del sistema, incluyendo títulos de películas, valoraciones, información básica de usuarios y es accedida por el backend y el pipeline de datos mediante una capa de acceso a datos para garantizar una gestión eficiente y consistente de la información.

Servicios ML

- El servicio de Machine Learning es el núcleo del sistema de recomendación, su función es inferir las recomendaciones de películas a partir del modelo entrenado, exponiendo un endpoint REST para la inferencia y devolviendo resultados personalizados que son consumidos por el backend.

Pipeline de datos

- El pipeline de datos garantiza un procesamiento reproducible de la información, este componente se encarga de la ingesta de datasets públicos, limpieza y transformación de datos, construcción de la matriz usuario-película y exportación de datos procesado para el entrenamiento de modelo

Dashboards

- El dashboard analítico permitirá visualizar estadísticas del sistema, como distribución de valoraciones, comportamiento de los usuarios y ejemplos de recomendaciones generadas.

Otros servicios

- El backend se comunica con la API de TMDB para obtener información adicional de las películas inferidas por el sistema de recomendación, como imágenes, sinopsis y metadatos, y transmitirla al frontend con el fin de enriquecer la visualización de los resultados.

6. Tecnologías seleccionadas

Lenguajes:

- Python, HTML, JS, CSS, SQL.

Frameworks:

- **Fast API:** Se ha elegido FastAPI por su alto rendimiento, soporte para asincronía y facilidad para exponer modelos de machine learning mediante endpoints REST.
- **Streamlit:** Se propone el uso de Streamlit debido a su simplicidad y rapidez de desarrollo para proyectos académicos.
- **Pytest:** Sintaxis sencilla, que permite escribir pruebas complejas con menos código.

Librerías principales:

- **Pandas:** Manipulación, limpieza y transformación de datos estructurados durante el proceso ETL.
- **NumPy:** Para operaciones numéricas y manejo eficiente de arrays, utilizada como soporte para el procesamiento de datos.
- **Scikit-learn:** Librería de Machine Learning empleada para el entrenamiento, evaluación y validación de modelos.

Tipo de base de datos:

- **PostgreSQL:** Ofrece alto rendimiento en consultas estructuradas y es ampliamente utilizado en entornos profesionales. Además, por su compatibilidad con herramientas de análisis y APIs modernas.

Herramientas auxiliares:

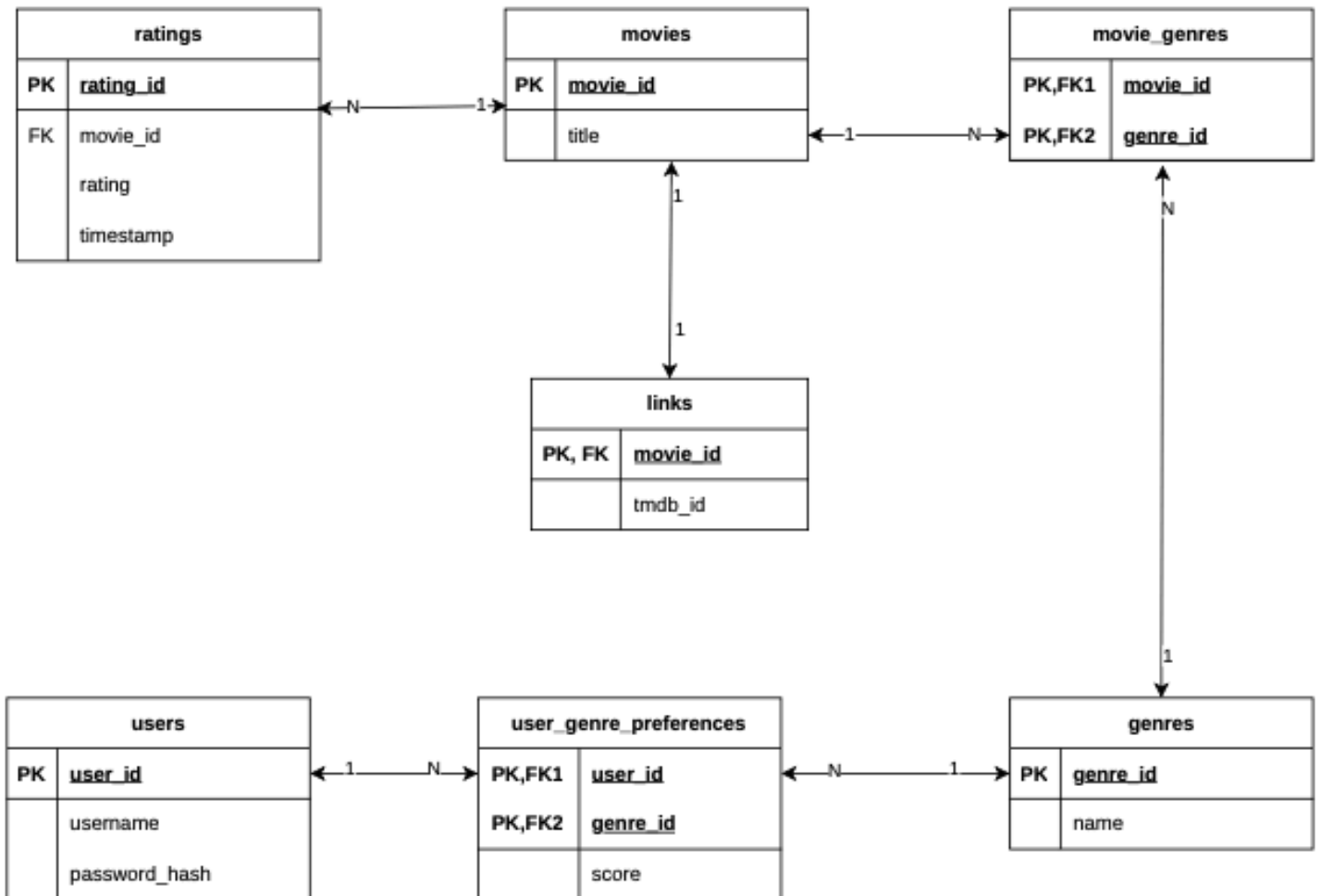
- **GitHub:** Control de versiones y trabajo colaborativo.
- **Trello:** Gestión de tareas y planificación de proyecto.
- **Visual:** Studio code: IDE de desarrollo.
- **Draw.io:** Presentación visual de arquitectura sistema.

Alternativa descartada (opcional) :

- Desarrollo completo del modelo sin librerías específicas.

7. Modelo de datos

Diagrama entidad-relación :



Explicación del diagrama :

El modelo de datos de CineMatch se organiza en dos bloques funcionales claramente diferenciados.

Bloque dataset (MovieLens): La tabla **ratings** es el núcleo del entrenamiento, contiene las 32 millones de valoraciones históricas con los campos movieid, rating y timestamp. Se relaciona con **movies** (N:1), que almacena los 87.585 títulos del catálogo. Los géneros se normalizan a través de la tabla de jointura **movie_genres**, que resuelve la relación N:M entre **movies** y **genres**, el catálogo controlado de 19 géneros cinematográficos. Por último, **links** se relaciona con **movies** en 1:1 y almacena el tmdb_id, utilizado para enriquecer la visualización con pósters y sinopsis obtenidos desde la API de TMDb.

Bloque plataforma (CineMatch): La tabla **users** almacena las credenciales de los usuarios registrados en la aplicación (username y password en formato hash). Se relaciona con **user_genre_preferences** (1:N), tabla de jointura que recoge las preferencias de género de cada usuario generadas en el formulario inicial, con un campo score (0.00–1.00) que representa la afinidad del usuario hacia cada género. Esta información se utiliza como solución al problema del cold start hasta que el modelo disponga de suficiente historial del usuario.

Diccionario de variables :

Campo	Tabla origen (CSV)	Tipo	Ejemplo	Descripción
user_id	Generado en DB	INTEGER	1, 42, 200948	ID anónimo del usuario de MovieLens. No corresponde a usuarios de CineMatch
movie_id	rating.csv/movies.csv/links.csv	INTEGER	1, 296, 87585	ID único de película, consistente entre todos los ficheros del dataset
rating	ratings.csv	FLOAT	0.5, 3.0, 5.0	Valoración en escala de 0.5 a 5.0 con incrementos de 0.5 estrellas
timestamp	ratings.csv	INTEGER	964982703	Momento del evento en Unix epoch (segundos desde 1970-01-01 UTC)
title	movies.csv	STRING	Toy Story (1995)	Título con año de estreno entre paréntesis. Puede contener errores menores
genre_id	movies.csv	STRING	Action Comedy	Lista de géneros separados por . Se normaliza en genres y la relación película-género en movie_genres .
tmdb_id	links.csv	INTEGER	862	ID en TMDb, usado para obtener pósters y sinopsis desde la API
username	users.csv	STRING	alejandro_99	Nombre de usuario único para login
password_hash	users.csv	STRING	\$2b\$12\$JH...	Hash de la contraseña
score	user_genre_preferences	FLOAT	0.75, 1.2	Peso/preferencia del usuario
name	genres.csv	STRING	Action, Comedy	Nombre legible del género

8. Datos

8.1 Fuente o fuentes :

Nombre del dataset: MovieLens 32M (ml-32m)

Proveedor: GroupLens Research — Universidad de Minnesota

URL de descarga: <https://grouplens.org/datasets/movielens/32m/>

Categoría oficial: Recommended for new research

Formato de descarga: ml-32m.zip (239 MB)

8.2 Licencia:

El dataset está disponible de forma pública bajo las siguientes condiciones establecidas por la Universidad de Minnesota:

- Uso permitido para investigación y proyectos académicos sin coste.
- Se debe citar la publicación oficial en cualquier trabajo que use el dataset: Harper & Konstan (2015), The MovieLens Datasets: History and Context. ACM TiiS 5(4).
- Se permite redistribuir el dataset con transformaciones, siempre bajo las mismas condiciones de licencia.
- Uso comercial prohibido sin autorización previa del GroupLens Research Project.
- La Universidad de Minnesota no garantiza la exactitud de los datos ni su idoneidad para ningún propósito específico.

El uso en CineMatch es estrictamente académico y cumple íntegramente con estos términos

8.2 Tipo de datos :

Los datos son de tipo tabular, distribuidos en 4 ficheros CSV codificados en UTF-8 con cabecera en la primera fila:

Archivo	Campos	Descripción
ratings.csv	userId, movieId, rating, timestamp	32.000.204 valoraciones de 0.5 a 5.0 (escala de 0.5 estrellas)
movies.csv	movieId, title, genres	87.585 películas con título y géneros (pipe-separated)
links.csv	movieId, imdbId, tmdbId	IDs externos para enriquecer con TMDb e IMDb
tags.csv	userId, movieId, tag, timestamp	2.000.072 etiquetas libres asignadas por usuarios

- Los géneros en movies.csv son una lista separada por el carácter '|' (pipe), seleccionados de un vocabulario controlado de 19 categorías: Action, Adventure, Animation, Children's, Comedy, Crime, Documentary, Drama, Fantasy, Film-Noir, Horror, Musical, Mystery, Romance, Sci-Fi, Thriller, War, Western y (no genres listed).

- Los timestamps en ratings.csv y tags.csv representan segundos transcurridos desde el 1 de enero de 1970 UTC (formato Unix epoch).

8.3 Volumen estimado :

Métrica	ml-32m
Total de ratings	32.000.204
Películas	87.585
Usuarios	200.948
Tag applications	2.000.072
Tamaño comprimido	239 MB (ml-32m.zip)
Periodo de datos	09/01/1995 – 12/10/2023
Fecha de generación	13 de octubre de 2023
Mínimo de ratings por usuario	20 ratings

- Nota sobre el volumen: Dado que el dataset completo contiene aproximadamente 32 millones de registros, durante las fases de desarrollo y prototipado del modelo se empleará un submuestreo estratificado con el objetivo de reducir los tiempos de entrenamiento e iteración. Una vez estabilizado el diseño del modelo, se realizará el entrenamiento final utilizando el dataset completo, que también se empleará en la fase de evaluación final para obtener métricas representativas del rendimiento del sistema.

8.4 Frecuencia de actualización:

- El dataset ml-32m es estático — fue generado el 13 de octubre de 2023 y no recibirá actualizaciones futuras. Los datos cubren el periodo comprendido entre el 9 de enero de 1995 y el 12 de octubre de 2023.

8.5 Calidad esperada:

- Alta — MovieLens 32M es un benchmark de referencia en la investigación académica sobre sistemas de recomendación, con las siguientes garantías de calidad:

- Sin valores nulos en los campos principales (userId, movieId, rating, timestamp).
- Escala de valoración consistente: valores de 0.5 a 5.0 en incrementos de 0.5 estrellas.
- Todos los usuarios incluidos han valorado un mínimo de 20 películas, garantizando un perfil mínimo de preferencias.
- IDs de usuario y película consistentes entre todos los ficheros del dataset.
- Formato limpio, estandarizado y bien documentado mediante README oficial.
- Posibles inconsistencias menores en los títulos de películas (advertidas en el README), mitigables usando el movieId como clave primaria.

Se realizará un Análisis Exploratorio de Datos (EDA) en la fase inicial para confirmar la calidad, detectar outliers, analizar la distribución de valoraciones y caracterizar el comportamiento de usuarios y películas.

8.6 Riesgos de datos (faltantes, sesgos, ruido...)

Riesgo	Descripción	Mitigación
Cold start	Películas con pocas valoraciones dificultan la recomendación personalizada	Recomendación por popularidad como fallback + formulario inicial de preferencias
Sesgo de popularidad	Las películas más conocidas acumulan la mayoría de ratings, sesgando el modelo hacia títulos populares	Diversificación en la generación del Top-K; penalizar ítems sobre-representados
Datos desbalanceados	Alta concentración de ratings en pocos usuarios muy activos	Aplicar umbral mínimo de interacciones por usuario y película durante el preprocesado
Volumen de datos	32M de registros pueden generar problemas de memoria y tiempo de entrenamiento con scikit-learn	Submuestreo estratificado para prototipado; uso del dataset completo solo en evaluación final
Dataset estático	No incluye películas posteriores a octubre de 2023 ni actualizaciones en tiempo real	Enriquecimiento con TMDb API para obtener metadata e imágenes actualizadas en la visualización
Ruido en ratings	Valoraciones inconsistentes o introducidas con poca fiabilidad por parte de algunos usuarios	Normalización de valoraciones por usuario; análisis de outliers en el EDA previo
Inconsistencias en títulos	El README advierte que los títulos pueden contener errores e inconsistencias	Uso de moviedl como clave primaria; validar títulos contra TMDb API

Son reales: Los ratings y tags provienen de interacciones reales de usuarios de la plataforma MovieLens durante casi 29 años de actividad.

Son anónimos: Los identificadores de usuario son numéricos aleatorios, sin ninguna información demográfica ni dato personal identificable (sin PII). Los usuarios fueron seleccionados aleatoriamente para su inclusión.

No son simulados: No se utilizan datos sintéticos ni mockeados para el entrenamiento del modelo de recomendación.

Excepción (datos de prueba): Los datos de usuarios creados durante las pruebas de la plataforma (registro, preferencias) serán mockeados a efectos de demo, pero no se usarán para el entrenamiento del modelo.

9. Ética, legalidad y accesibilidad

Apartado obligatorio:

Presencia o ausencia de datos personales (PII) :

El sistema **no maneja datos personales reales (PII)**. La fuente de datos principal es MovieLens, un dataset público con valoraciones completamente anónimas donde los usuarios están identificados únicamente por IDs numéricos sin ningún dato identificativo asociado (sin nombre, email, edad, ubicación, etc.).

En cuanto a los usuarios registrados en la plataforma propia, el sistema únicamente almacena:

- **username** (elegido libremente por el usuario, no necesariamente real)
- **password_hash** (contraseña hasheada, nunca en texto plano)

No se recoge ni almacena ningún otro dato personal: no hay emails, fechas de nacimiento, IPs persistentes ni información de navegación.

Cumplimiento GDPR :

Dado que el sistema solo gestiona usuarios propios con datos mínimos (username + hash de contraseña), el impacto regulatorio bajo el GDPR es bajo. No obstante, se contemplan las siguientes medidas de cumplimiento:

Minimización de datos : Solo se recogen username y password_hash. Sin emails ni datos personales adicionales.

Finalidad : Los datos del usuario se usan exclusivamente para autenticación y personalización de recomendaciones.

Seguridad : Las contraseñas se almacenan hasheadas.

Derecho al olvido : Se contempla un endpoint de borrado de cuenta y datos asociados.

Consentimiento : Al registrarse, el usuario acepta el uso de sus valoraciones para generar recomendaciones.

Dataset externo : MovieLens es de uso académico libre; no contiene PII según su licencia pública

Riesgos de sesgo :

El sistema de recomendación por filtrado colaborativo presenta los siguientes riesgos de sesgo identificados:

1. Popularity bias (sesgo de popularidad) Las películas con muchas valoraciones tienen mayor peso en el modelo, lo que tiende a recomendar siempre el mismo conjunto de títulos populares, perjudicando películas con nicho pero alta calidad.

2. Cold start (arranque en frío)

- *Usuario nuevo*: sin historial de valoraciones, el modelo no tiene base para generar recomendaciones personalizadas.
- *Película nueva o poco valorada*: infrarepresentada en la matriz usuario-película, casi nunca recomendada.

3. Sesgo de confirmación (filter bubble) El filtrado colaborativo tiende a recomendar contenido similar a lo ya valorado, reduciendo la diversidad y encerrando al usuario en una burbuja de sus propios gustos.

4. Sesgo demográfico implícito Aunque MovieLens es anónimo, patrones culturales o de género pueden estar implícitamente reflejados en las valoraciones históricas, heredándose al modelo sin que sea visible.

Medidas de mitigación :

Popularity bias : Incorporar métricas de diversidad en la evaluación (más allá de RMSE/Precision@K). Penalizar explícitamente la sobreexposición de títulos ya muy populares.

Cold start usuario : Sistema de recomendación baseline por popularidad activo desde el inicio. Opcionalmente, onboarding con selección de géneros favoritos usando `user_genre_preferences`.

Cold start película : Usar metadatos de género (tabla `genres`) para recomendar por contenido cuando falten valoraciones.

Filter bubble : Introducir un factor de exploración (serendipity): mezclar recomendaciones personalizadas con algunas sugerencias de géneros poco explorados por el usuario.

Sesgo implícito : Documentar el origen y limitaciones del dataset. Evaluar la distribución de géneros recomendados para detectar infrarrepresentación.

Accesibilidad básica prevista:

El frontend está desarrollado en **Streamlit**, que proporciona una base de accesibilidad razonable por defecto. Se contemplan las siguientes medidas mínimas:

- **Contraste de colores**: uso de paletas con contraste suficiente para usuarios con dificultades visuales (ratio mínimo WCAG AA).
- **Navegación por teclado**: Streamlit soporta navegación básica por teclado de forma nativa.
- **Textos descriptivos**: todas las visualizaciones y gráficos incluirán títulos y etiquetas explicativas (sin depender únicamente del color para transmitir información).
- **Mensajes de error claros**: el sistema informará al usuario con mensajes legibles cuando no haya recomendaciones disponibles (ej. cold start).
- **Responsive básico**: Streamlit adapta el layout a distintos tamaños de pantalla por defecto.

El proyecto no contempla en esta fase una auditoría de accesibilidad formal, pero se adoptan buenas prácticas desde el diseño inicial.

410. Backlog inicial

Entre 8 y 12 historias de usuario.

HU-01 · Registro de usuario

Como usuario nuevo, quiero poder registrarme con un nombre de usuario y contraseña, para tener acceso personalizado a las recomendaciones.

Criterios de aceptación:

- El sistema acepta username + contraseña y crea una cuenta.
 - La contraseña se almacena hasheada en BBDD (nunca en texto plano).
 - Si el username ya existe, se muestra un mensaje de error claro.
 - El registro exitoso redirige al usuario a la pantalla principal.
-

HU-02 · Inicio de sesión

Como usuario registrado, quiero poder iniciar sesión con mis credenciales, para acceder a mis recomendaciones personalizadas.

Criterios de aceptación:

- El sistema valida username y contraseña contra la BBDD.
 - Las credenciales incorrectas muestran un mensaje de error sin revelar cuál campo es incorrecto.
 - El inicio de sesión correcto genera una sesión activa y redirige al panel principal.
-

HU-03 · Recomendaciones personalizadas

Como usuario autenticado con historial de valoraciones, quiero recibir una lista de películas recomendadas para mí, para descubrir contenido que se ajuste a mis gustos.

Criterios de aceptación:

- El sistema devuelve un Top-K (mínimo 10) de películas no vistas/valoradas por el usuario.
- Las recomendaciones se generan mediante el modelo de filtrado colaborativo entrenado.
- El tiempo de respuesta de la API no supera los 3 segundos.
- Se muestra el título de la película y, si está disponible, su poster (vía TMDB API).

HU-04 · Recomendaciones para usuario nuevo (cold start)

Como usuario nuevo sin historial de valoraciones, quiero recibir sugerencias de películas, para no encontrarme con una pantalla vacía al entrar por primera vez.

Criterios de aceptación:

- Si el usuario no tiene valoraciones, el sistema devuelve las películas más populares/mejor valoradas del dataset (baseline por popularidad).
 - Se indica visualmente al usuario que las recomendaciones son generales y mejorarán con sus valoraciones.
-

HU-05 · Valorar una película

Como usuario autenticado, quiero poder puntuar una película que he visto, para que el sistema mejore mis futuras recomendaciones.

Criterios de aceptación:

- El usuario puede asignar una puntuación de 0.5 a 5 (en incrementos de 0.5, siguiendo el esquema MovieLens).
 - La valoración se persiste correctamente en la BBDD asociada al usuario.
 - Una valoración ya existente puede actualizarse.
 - Tras valorar, el sistema confirma la acción con feedback visual.
-

HU-06 · Búsqueda de películas

Como usuario, quiero poder buscar películas por título o género, para encontrar una película concreta o explorar un género de mi interés.

Criterios de aceptación:

- El buscador filtra resultados en tiempo real (o al pulsar buscar) por título parcial o género.
 - Los resultados muestran título, género(s) y puntuación media.
 - Si no hay resultados, se muestra un mensaje informativo.
-

HU-07 · Consultar detalle de una película

Como usuario, quiero ver información detallada de una película (sinopsis, género, puntuación media), para decidir si me interesa verla.

Criterios de aceptación:

- Al seleccionar una película se muestra: título, género(s), puntuación media del sistema y (si disponible vía TMDB) poster y sinopsis.
 - La información de TMDB se recupera usando el `tmdb_id` de la tabla `links`.
 - Si TMDB no está disponible, se muestran al menos los datos locales sin error.
-

HU-08 · Ver mis preferencias de género

Como usuario autenticado, quiero ver qué géneros prefiero según mi historial, para entender mejor mis gustos y el comportamiento del sistema.

Criterios de aceptación:

- El sistema muestra una visualización (gráfico de barras o similar) con los géneros más presentes en las películas bien valoradas por el usuario.
 - Los datos se extraen de `user_genre_preferences`.
 - La visualización es comprensible sin necesidad de explicación adicional.
-

HU-09 · Modificar mis preferencias de género

Como usuario autenticado, quiero modificar qué géneros prefiero para que así se tengan en cuenta a la hora de mostrarme recomendaciones..

Criterios de aceptación:

- El sistema muestra una selección de los géneros seleccionados previamente por el usuario y ofrece la posibilidad de modificarlos.
- Los datos se extraen de `user_genre_preferences`.
- La visualización y las opciones de modificación son comprensibles sin necesidad de explicación adicional.

11. Plan de trabajo

Estructura de Reuniones Semanales :

Se mantienen 3 reuniones fijas por semana, tal y como se acuerda en el equipo. El canal principal es Discord con respaldo en Telegram.

Reunion	Día / Hora	Canal	Objetivo
Reunión 1 (inicio semana)	Lunes -:00h	Discord / Telegram	Revisión de tareas pendientes, asignación de issues en Trello, desbloqueo de impedimentos
Reunion 2 (mid-week)	Miércoles -:00h	Discord / Telegram	Seguimiento de progreso, revisión de PRs abiertos, decisión sobre cambios de diseño
Reunión 3 (cierre semana)	Domingo -:00h	Discord / Telegram	Retrospectiva semanal, demo de avances, planificación siguiente semana, actualización Trello

Hitos principales :

El proyecto se divide en 8 semanas con hitos claros al final de cada una para asegurar progreso continuo y detección temprana de desviaciones.

Semana	Hito	Entregable	Responsable
S1	Entorno listo + datos descargados	Dataset ingestado, repo configurado, app.py base en Streamlit	Data + Todos
S2	EDA completo + BBDD operativa	Notebook EDA, esquema BBDD, API auth funcional, baseline ML	Data + Backend + ML
S3	Pipeline de datos + modelo colaborativo v1	Pipeline reproducible, primer modelo CF entrenado, frontend con formulario	Data + ML + Frontend
S4	Integración backend-modelo	Endpoints de recomendación operativos, ajuste de métricas iniciado	Backend + ML
S5	Sistema integrado end-to-end	Sistema completo: usuario puede registrarse, valorar y recibir recomendaciones	Todos
S6	Dashboard + TMDB integrado	Dashboard con métricas, posters via TMDB, documentación técnica	Frontend + Backend
S7	QA + ajustes finales	Tests de integración pasados, bugs críticos resueltos, doc completa	Todos
S8	Entrega y demo final	Presentación, demo funcional, repositorio final entregado	Todos

Cronograma Gantt por Semanas

Distribución semanal de tareas por rol. Las celdas coloreadas indican semanas activas para cada tarea.

Tarea	Rol	S1	S2	S3	S4	S5	S6	S7	S8
Ingesta dataset MovieLens	Data								
Limpieza y EDA	Data								
Pipeline reproducible datos	Data								
Documentación datos	Data								
Modelo baseline (popularidad)	ML								
Diseño filtrado colaborativo	ML								
Entrenamiento y evaluación	ML								
Ajuste métricas (P@K, R@K)	ML								
Documentación modelo	ML								
Diseño BBDD (esquema)	Backend								
API REST FastAPI (auth)	Backend								
Integración modelo + API	Backend								
Endpoints recomendaciones	Backend								
Integración TMDB API	Backend								
Documentación API	Backend								
App Streamlit básica	Frontend								
Formulario / búsqueda	Frontend								
Visualización top-K recom.	Frontend								
Dashboard estadísticas	Frontend								
Refinamiento UI/UX	Frontend								
Pruebas de integración	Todos								
Demo y presentación final	Todos								

Dependencias entre tareas :

Mapa de dependencias críticas del proyecto. Un retraso en cualquier tarea listada en 'Depende de' bloqueara directamente las tareas indicadas en 'Bloquearia a'.

Tarea	Depende de	Bloquearía a
Pipeline de datos	Ingesta y limpieza de datos (Data)	Entrenamiento del modelo ML
Modelo ML v1	Pipeline de datos listo	Integración Backend-ML, endpoints de recomendación
BBDD operativa	Diseño del esquema (Backend S1)	API REST, autenticación, valoraciones de usuario
API REST (endpoints recom.)	Modelo ML integrado + BBDD operativa	Frontend (visualización top-K, dashboard)
Integración TMDB	Endpoints de película en API (movie_id -> tmdb_id)	Visualización de posters y sinopsis en Frontend
Dashboard estadísticas	Métricas de modelo disponibles + API operativa	Demo final
Pruebas de integración	Sistema end-to-end completo (S5)	Entrega final

Reparto de tareas :

Cada integrante tiene un rol principal pero se compromete a apoyar en casos de bloqueo según el acuerdo de trabajo del equipo.

Integrante	Rol	Responsabilidades principales	Herramienta
Badr Lahrarti	Data Engineer	Ingesta MovieLens, limpieza, EDA, pipeline reproducible, Git lead, doc datos	Pandas, NumPy, GitHub
Alejandro Gonzalez	ML Engineer	Baseline popularidad, filtrado colaborativo, entrenamiento, Precision@K, Recall@K, doc modelo	scikit-learn, Python
Daniel Ruiz	Backend Dev	API REST FastAPI, autenticación, BBDD, integración modelo, TMDB API, doc endpoints	FastAPI, SQLAlchemy
Fran Lopez	Frontend / BI	Interfaz Streamlit, formularios, top-K visualización, dashboard métricas, doc UI	Streamlit, Plotly

Tabla Trello :

<https://trello.com/invite/b/6998875b4f1f5eb63aa51e76/ATTI1080de59ebe66868d3d467497cd0652168BDF5B0/ifp-grupo-e-12>

Riesgos técnicos :

Análisis de los principales riesgos técnicos identificados, con su probabilidad, impacto y las medidas de mitigación y plan B definidos.

Riesgo técnico	Prob.	Impacto	Mitigación	Plan B
Cold start: usuarios/películas sin valoraciones	Alta	Alto	Recomendación por géneros (user_genre_preferences)	Baseline por popularidad activado desde S1
Métricas ML insuficientes (RMSE alto, P@K bajo)	Media	Alto	Ajuste de hiperparametros, prueba de distintos algoritmos CF	Modelo baseline por popularidad como entregable minimo
Escalabilidad: lentitud con dataset grande	Media	Medio	Usar subset del dataset (100k ratings) para desarrollo	Limitar recomendaciones a top películas populares
TMDB API inaccesible o rate-limit	Baja	Bajo	Caché de respuestas TMDB en BBDD local	Mostrar solo datos locales (titulo, genero, rating)
Pipeline demasiado complejo / falta de tiempo	Media	Alto	Definir MVP claro: carga -> modelo -> API minimal	Eliminar dashboard y TMDB, entregar solo recomendaciones
Falta de disponibilidad de un integrante	Baja	Medio	Tareas documentadas en Trello, trabajo asincrónico en Git	Redistribución temporal de tareas entre el equipo

Acuerdo de Trabajo del Equipo :

- **Canal de comunicación:**
- Canal de comunicación: Discord (principal) + Telegram (respaldo)
- Horario de disponibilidad: Lunes, Miércoles y Domingo a partir de las 18:00h
- Frecuencia de reuniones: 3 días / semana (Lunes, Miércoles, Domingo)
- Regla 1: No se sube nada a main sin Pull Request aprobado
- Regla 2: Todo trabajo tiene issue asignada en Trello / GitHub
- Regla 3: Las decisiones técnicas importantes se documentan
- Regla 4: Se respeta el reparto de roles pero se ayuda si hay bloqueo
- Repositorio: github.com/badrlah9/CineMatch-Sistema-de-recomendacion-peliculas

12. Estrategia de desarrollo

Flujo Git (branches)

De momento hay 2 branches main, develop.

Cómo se integrará el código :

Cómo se validará funcionalidad

Cómo se hará testing básico : El proyecto utilizará **pytest** como framework principal de testing. Cada capa del sistema tendrá sus propios tests unitarios. La integración se validará mediante pruebas end-to-end manuales antes de cada entrega. El modelo ML se evaluará mediante split 80/20 con métricas RMSE y Precision@K.

13. Riesgos técnicos actualizados

Riesgo: Escalabilidad y volumen de datos (32M de registros).

- **Impacto:** Medio.
- **Probabilidad:** Media.
- **Plan de mitigación:** Uso de submuestreo estratificado para desarrollo y limitación a películas populares en caso de lentitud.

Riesgo: Cold start (usuarios/películas nuevas).

- **Impacto:** Alto.
- **Probabilidad:** Alta.
- **Plan de mitigación:** Recomendación por popularidad (*fallback*) y uso del formulario inicial de preferencias de género.

Riesgo: Métricas del modelo insuficientes (RMSE alto/Precisión baja).

- **Impacto:** Alto.
- **Probabilidad:** Media.
- **Plan de mitigación:** Ajuste de hiperparámetros y prueba de diversos algoritmos de filtrado colaborativo.

Riesgo: Inestabilidad de la API externa (TMDB).

- **Impacto:** Bajo.
- **Probabilidad:** Baja.
- **Plan de mitigación:** Implementación de caché local en PostgreSQL y mostrar datos locales como respaldo si falla la conexión.

Riesgo: Sesgo de popularidad y burbuja de filtro.

- **Impacto:** Medio.
- **Probabilidad:** Alta.
- **Plan de mitigación:** Aplicar métricas de diversidad y penalizar la sobreexposición de títulos muy populares en el Top-K.

14. Repositorio del proyecto

<https://github.com/badrlah9/CineMatch-Sistema-de-recomendacion-peliculas>